SOFTWARE REENGINEERING

ASSIGNMENT#2

Samavia safdar
Numl-f22-18262

Hania Abrar
Numl-f22-16830

Bushra Mubeen
Numl-f22-23380

SUBMITTED TO:

Miss Saiqa Anjum

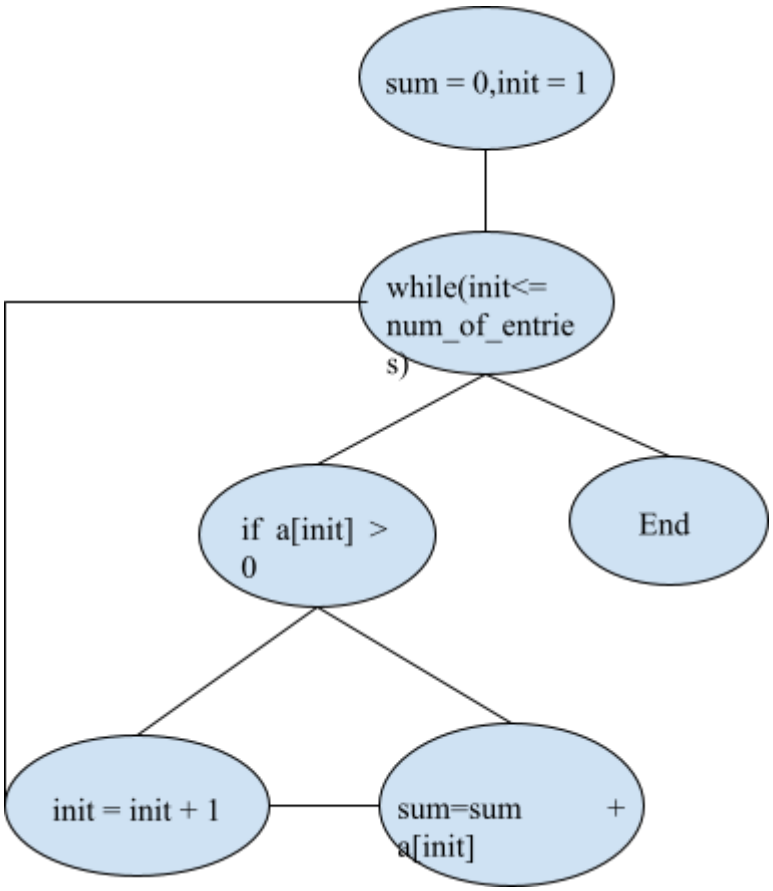
Assignment 02[CLO2]

SOFTWARE RE ENGINEERING

Task1-

Design a CFG for the following sample code. Determine the cyclomatic complexity of the graph.

(a) sum_of_all_positive_numbers(a, num_of_entries, sum)
(b) sum = 0
(c) init = 1
(d) while(init <= num_of_entries)
(e) if a[init] > 0
(f) sum = sum + a[init] endif
(g) init = init + 1 endwhile
(h) end sum_of_all_positive_number



CYCLOMATIC COMPLEXITY:

Formula:

$$CC=E-N+2$$

E = number of Edges = 7

N = Number of Nodes = 6

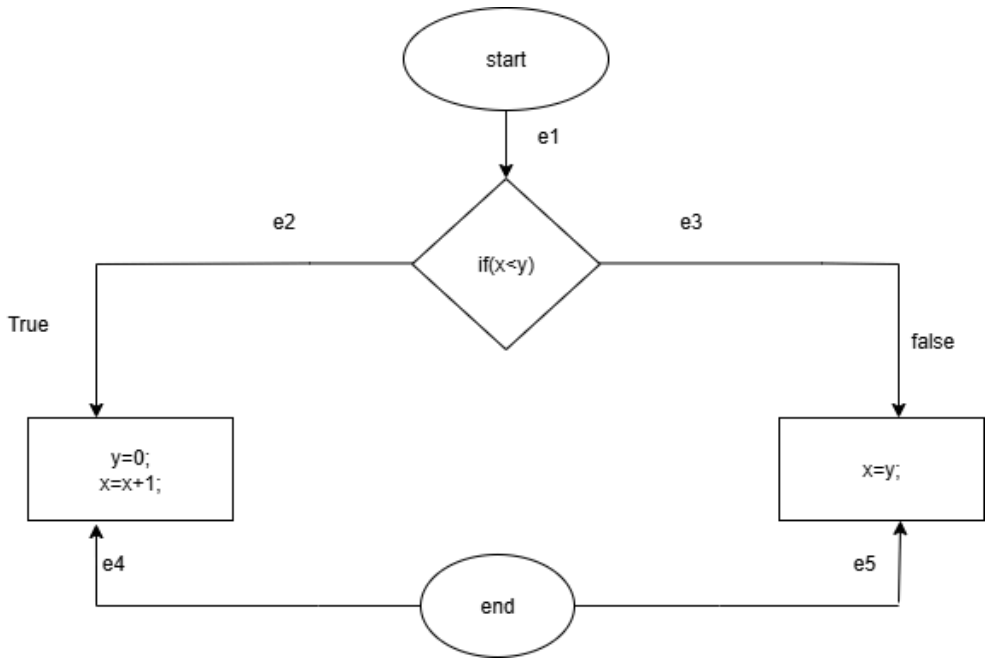
$$CC = 7-6+2$$

$$CC = 3$$

Task2-Draw graph and label edges also

```
if (x < y)
{
    y = 0;
    x = x +
1;
}
else
{
    x = y;
}
```

<u>CFG GRAPH:</u>



Task3-

Slice the program with respect to Chars, words and lines and words**.**

Original Program

```
inword = F
chars = 0
lines = 0
words = 0
read(c)
while(c != EOF)
{
  chars++
  if (c == '\n')
    lines++
  if (isletter(c)))
  {
    if (!inword)
    {
      words++
      inword = T
    }
  }
  else
    inword = F
  read(c)
}
```

Slice the given program with respect to chars, words, and lines and words.

- **Original Program**

```
inword = F
chars = 0
lines = 0
words = 0
read(c)
while (c != EOF)
{
   chars++
   if (c == '\n')
      lines++
   if (isletter(c))
   {
      if (!inword)
      {
         words++
         inword = T
      }
   }
   else
      inword = F
   read(c)
}
```

Program Slice with respect to chars

When slicing with respect to chars, we only keep the statements that **affect or depend on** the variable chars.

Sliced Program (chars):

```
chars = 0
read(c)
while (c != EOF)
{
   chars++
   read(c)
}
```

Explanation:

Only the statements that count characters are retained.
All other logic (lines, words, conditions) is removed.

Program Slice with respect to words

Now, we keep the statements that affect the variable words.
Since words depends on whether the current character is a letter and whether we are already *in a word*, we also keep statements for inword.

Sliced Program (words):

```
inword = F
words = 0
read(c)
while (c != EOF)
{
   if (isletter(c))
   {
      if (!inword)
      {
         words++
         inword = T
      }
   }
   else
      inword = F
   read(c)
}
```

Explanation:
words is incremented only when a new word begins (when !inword and the current character is a letter).
Hence, inword logic is essential.

Program Slice with respect to lines and words

Now, we are slicing for both lines and words.
So, we keep all statements that affect **either** variable.

Sliced Program (lines and words):

```
inword = F
   lines = 0
   words = 0
   read(c)
   while (c != EOF)
   {
      if (c == '\n')
         lines++
      if (isletter(c))
      {
         if (!inword)
         {
            words++
            inword = T
         }
      }
      else
         inword = F
      read(c)
   }
```

Explanation:
This version retains the statements for counting both lines (lines++) and words (words++), along with the logic for inword.